claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-aa4a5478b6e9f5947.jsonl`
- SHA-256: `007d2493446825edb601309bcb69f46cf71d8cd886a539a660279f55bd6ced1b`
- Source modified: `2026-06-10T06:13:45+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:09:28.015Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("collector") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"Licensing conflation: dataset license applied to broadcast video footage, with a default-MIT ingest flag","severity":"critical","area":"licensing / export gate","file":"C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py:12","evidence":"ingest.py:12 `parser.add_argument(\"--license\", default=\"MIT\", ...)` ? the CoachAI/ShuttleSet ingest defaults the license to MIT. The single `license_type` column on `videos` (db.py:63) covers the VIDEO, and `is_commercial` gates export (curate.py:478-480). The real committed export `data/eval_export/manifest.json` shows shuttleset_21 = 'An Se Young Ratchanok Intanon YONEX Thailand Open 2021' (a YouTube broadcast, video_url=youtube.com/watch?v=gloiZ_gTJaE) with license_type=\"MIT\", is_commercial=true. The MIT license belongs to the ShuttleSet annotation CSVs, not the broadcast footage.","why_it_matters":"The export's 'commercial-only' gate ? the property PIPELINE_ROLE.md says makes this corpus trainable for the owned model ? is fed by a defaulted/annotation-dataset license stamped onto video footage whose rights are almost certainly not MIT. 24 videos / 1915 segments are currently exported as commercially-safe on this basis. This directly undermines the repo-level guardrail that training data must be commercially clear."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:09:38.039Z)

```
1       import argparse
2       import sys
3       from src.core.config import load_config, get_database_path
4       from src.core.db import SportsDatabase
5       from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7       def main():
8           parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9           parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10          parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11          parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12          parser.add_argument("--license", default="MIT", help="Dataset license type")
13          parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
14          parser.add_argument("--match-name", help="Name of the match")
15          parser.add_argument("--video-id", help="Video unique identifier")
16
17          args = parser.parse_args()
18
19          # Load db path from the central config (falls back to the default).
20          # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
21          # sports are populated via `curate import-local` or vendor golden sets.
22          db = SportsDatabase(get_database_path(load_config()))
23
24          if args.sport == "badminton" and args.parser == "coachai":
25              match_parser = CoachAIBadmintonParser()
26              try:
27                  video_meta, rallies = match_parser.parse(
28                      args.file_path,
29                      video_id=args.video_id,
30                      video_url=args.video_url,
31                      license=args.license,
32                      match_name=args.match_name
33                  )
34
35                  # Insert into database (by default parsed entries are marked as staging)
36                  db.insert_video(video_meta)
37                  count = db.insert_badminton_rallies(rallies)
38
39                  print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
40                  print(f"  - Video ID: {video_meta.video_id}")
41                  print(f"  - Total Rallies: {count}")
42                  print(f"  - License: {video_meta.license_type.value}")
43                  print(f"  - Commercial safe: {video_meta.is_commercial}")
44              except Exception as e:
45                  print(f"Error parsing file: {e}")
46                  sys.exit(1)
47
48      if __name__ == "__main__":
49          main()
50
```

3. user (2026-06-10T06:09:38.801Z)

C:\Users\avidu\Projects\sports-data-collector\src\core\db.py

4. user (2026-06-10T06:09:39.104Z)

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py

5. user (2026-06-10T06:09:44.922Z)

```
1       import sqlite3
2       import os
3       from contextlib import contextmanager
4       from typing import List, Dict, Any, Optional
5       from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6       from src.core.schemas.badminton import BadmintonRally
7       from src.core.schemas.tennis import TennisPoint
8       from src.core.schemas.cricket import CricketDelivery
9       from src.core.schemas.pickleball import PickleballRally
10      from src.core.schemas.tabletennis import TableTennisRally
11
12      # Maps each sport to its annotation table and the column(s) that order segments
13      # chronologically. All child tables expose (start_time, end_time), so a
14      # single generic reader can serve every sport ? adding a new sport is one entry
15      # here plus its CREATE TABLE, with no new query code.
16      SEGMENT_TABLES = {
17          SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18          SportType.TENNIS: ("tennis_points", "point_number"),
19          SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20          SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21          SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22      }
23
24
25      class SportsDatabase:
26          def __init__(self, db_path: str):
27              self.db_path = db_path
28              # Ensure target directory exists
29              os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30              self._init_db()
31
32          @contextmanager
33          def _get_connection(self):
34              """Yield a SQLite connection and always close it.
35
36              Used as ``with self._get_connection() as conn:``. A plain
37              ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38              transaction but does NOT close the connection, which leaks handles and
39              raises ResourceWarnings. This wrapper guarantees the connection is closed
40              on exit; writers still commit explicitly inside the block.
41              """
42              conn = sqlite3.connect(self.db_path)
43              conn.row_factory = sqlite3.Row
44              # SQLite disables foreign keys per-connection by default; enable so the
45              # ON DELETE CASCADE declarations on the child tables actually fire.
46              conn.execute("PRAGMA foreign_keys = ON")
47              try:
48                  yield conn
49              finally:
50                  conn.close()
51
52          def _init_db(self):
53              with self._get_connection() as conn:
54                  cursor = conn.cursor()
55
56                  # 1. Create common videos table
57                  cursor.execute("""
58                  CREATE TABLE IF NOT EXISTS videos (
59                      video_id TEXT PRIMARY KEY,
60                      sport TEXT NOT NULL,
61                      video_url TEXT,
62                      local_path TEXT,
63                      license_type TEXT NOT NULL,
```

```
                    license_url TEXT,
                    is_commercial INTEGER NOT NULL DEFAULT 0,
                    status TEXT NOT NULL DEFAULT 'staging',
                    match_name TEXT NOT NULL,
                    fps REAL,
                    resolution TEXT
                )
                """)

                # 2. Create badminton_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS badminton_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 3. Create tennis_points table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tennis_points (
                    video_id TEXT NOT NULL,
                    point_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    set_score TEXT,
                    game_score TEXT,
                    server TEXT,
                    ending_type TEXT,
```

6. user (2026-06-10T06:09:45.437Z)

```
        def export_eval_set(self, args):
            """Export curated annotations as indexer-ready eval/training sets.

            Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
            indexer's `start,end` ground-truth format (decimal seconds), plus a single
            `<out_dir>/manifest.json` that pairs each CSV with its video file. The
            manifest is the contract the downstream eval/training driver reads: the
            indexer keys videos by *file MD5*, not by our `video_id`, so it must run
            against the exact `local_path`/`video_url` recorded here.

            By default only CURATED + commercially-safe videos are exported, mirroring
            the license gate already enforced elsewhere in the toolchain.
            """
            out_dir = args.out_dir
            commercial_only = not args.include_noncommercial
            statuses = [CurationStatus.CURATED]
            if args.include_staging:
                statuses.append(CurationStatus.STAGING)

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            manifest = []
            skipped_noncomm = 0
            skipped_empty = 0
```

```
465
466             for sport_str in sport_strs:
467                 try:
468                     sport = SportType(sport_str.lower())
469                 except ValueError:
470                     print(f"Warning: skipping unknown sport '{sport_str}'.")
471                     continue
472
473                 videos = []
474                 for status in statuses:
475                     videos.extend(self.db.get_videos_by_status(status, sport))
476
477                 for video in videos:
478                     if commercial_only and not video.is_commercial:
479                         skipped_noncomm += 1
480                         continue
481
482                     intervals = self.db.get_segment_intervals(video.video_id, sport)
483                     if not intervals:
484                         skipped_empty += 1
485                         continue
486
487                     sport_dir = os.path.join(out_dir, sport.value)
488                     os.makedirs(sport_dir, exist_ok=True)
489                     csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490                     with open(csv_path, "w", newline="", encoding="utf-8") as f:
491                         writer = csv.writer(f)
492                         writer.writerow(["start", "end"])
493                         for start, end in intervals:
494                             writer.writerow([start, end])
495
496                     manifest.append({
497                         "video_id": video.video_id,
498                         "sport": sport.value,
499                         "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500                         "local_path": video.local_path,
501                         "video_url": video.video_url,
502                         "match_name": video.match_name,
503                         "fps": video.fps,
504                         "resolution": video.resolution,
505                         "license_type": video.license_type.value,
506                         "is_commercial": video.is_commercial,
507                         "status": video.status.value,
508                         "num_segments": len(intervals),
509                     })
510
511             os.makedirs(out_dir, exist_ok=True)
512             manifest_doc = {
513                 "commercial_only": commercial_only,
514                 "statuses": [s.value for s in statuses],
515                 "total_videos": len(manifest),
516                 "total_segments": sum(m["num_segments"] for m in manifest),
517                 "eval_sets": manifest,
518             }
519             manifest_path = os.path.join(out_dir, "manifest.json")
```

7. user (2026-06-10T06:09:53.656Z)

```
No files found
```

8. user (2026-06-10T06:09:54.191Z)

```
No files found
```

9. user (2026-06-10T06:10:02.108Z)

```
C:\Users\avidu\Projects\sports-data-collector\data\eval_export\manifest.json
```

10. user (2026-06-10T06:10:03.613Z)

```
Found 5 files
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py
C:\Users\avidu\Projects\sports-data-collector\README.md
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py
C:\Users\avidu\Projects\sports-data-collector\src\crawlers\badminton_crawler.py
C:\Users\avidu\Projects\sports-data-collector\src\core\downloader.py
```

11. user (2026-06-10T06:10:04.232Z)

```
/c/Users/avidu/Projects/sports-data-collector/:
total 105
drwxr-xr-x 1 avidu 197609     0 Jun 10 11:35 .
drwxr-xr-x 1 avidu 197609     0 Jun  7 12:37 ..
-rw-r--r-- 1 avidu 197609 53248 Jun 10 11:35 .coverage
drwxr-xr-x 1 avidu 197609     0 Jun 10 11:09 .git
-rw-r--r-- 1 avidu 197609   264 May 26 01:26 .gitattributes
-rw-r--r-- 1 avidu 197609  1031 Jun  7 14:21 .gitignore
drwxr-xr-x 1 avidu 197609     0 May 25 23:59 .pytest_cache
drwxr-xr-x 1 avidu 197609     0 Jun  6 20:22 .sixth
-rw-r--r-- 1 avidu 197609  9583 Jun  2 22:52 README.md
drwxr-xr-x 1 avidu 197609     0 Jun  2 22:30 __pycache__
-rwxr-xr-x 1 avidu 197609  6802 Jun  2 22:36 cleanup.py
drwxr-xr-x 1 avidu 197609     0 May 26 01:31 config
drwxr-xr-x 1 avidu 197609     0 Jun  9 18:05 data
drwxr-xr-x 1 avidu 197609     0 Jun  8 16:39 docs
-rw-r--r-- 1 avidu 197609  1211 Jun  7 14:21 pyproject.toml
drwxr-xr-x 1 avidu 197609     0 Jun  7 13:10 src
drwxr-xr-x 1 avidu 197609     0 Jun  8 16:39 tests

/c/Users/avidu/Projects/sports-data-collector/data:
total 238
drwxr-xr-x 1 avidu 197609      0 Jun  9 18:05 .
drwxr-xr-x 1 avidu 197609      0 Jun 10 11:35 ..
drwxr-xr-x 1 avidu 197609      0 May 26 02:57 eval_export
-rw-r--r-- 1 avidu 197609    367 May 25 23:59 mock_local_badminton_anno.json
-rw-r--r-- 1 avidu 197609   1395 Jun  7 14:29 roora_badminton_canonical.csv
-rw-r--r-- 1 avidu 197609    827 Jun  7 14:29 roora_badminton_issues.txt
-rw-r--r-- 1 avidu 197609 200704 Jun  9 18:05 sports_metadata.db
-rw-r--r-- 1 avidu 197609    148 May 26 00:00 staging_coachai_match.csv
drwxr-xr-x 1 avidu 197609      0 Jun  2 22:36 temp
drwxr-xr-x 1 avidu 197609      0 Jun  7 14:29 validation
drwxr-xr-x 1 avidu 197609      0 May 26 02:49 videos
```

12. user (2026-06-10T06:10:12.153Z)

```
1       {
2         "commercial_only": true,
3         "statuses": [
4           "curated",
5           "staging"
6         ],
7         "total_videos": 24,
8         "total_segments": 1915,
9         "eval_sets": [
10          {
11            "video_id": "my_custom_local_match",
12            "sport": "badminton",
13            "csv": "badminton/my_custom_local_match.csv",
```

```
            "local_path": "data/videos/my_custom_match.mp4",
            "video_url": null,
            "match_name": "My Custom Local Match",
            "fps": 30.0,
            "resolution": "1080p",
            "license_type": "Proprietary",
            "is_commercial": true,
            "status": "curated",
            "num_segments": 2
          },
          {
            "video_id": "shuttleset_21",
            "sport": "badminton",
            "csv": "badminton/shuttleset_21.csv",
            "local_path": "./data/videos\\shuttleset_21.mp4",
            "video_url": "https://www.youtube.com/watch?v=gloiZ_gTJaE",
            "match_name": "An Se Young Ratchanok Intanon YONEX Thailand Open 2021
QuarterFinals",
            "fps": 30.0,
            "resolution": "1080p",
            "license_type": "MIT",
            "is_commercial": true,
            "status": "curated",
            "num_segments": 75
          },
          {
            "video_id": "shuttleset_1",
            "sport": "badminton",
            "csv": "badminton/shuttleset_1.csv",
            "local_path": "./data/videos\\shuttleset_1.mp4",
            "video_url": "https://www.youtube.com/watch?v=O669aZhH0LI",
            "match_name": "Kento MOMOTA CHOU Tien Chen Fuzhou Open 2019 Finals",
            "fps": 25.0,
            "resolution": "1080p",
            "license_type": "MIT",
            "is_commercial": true,
            "status": "staging",
            "num_segments": 113
          },
          {
            "video_id": "shuttleset_2",
            "sport": "badminton",
            "csv": "badminton/shuttleset_2.csv",
            "local_path": "./data/videos\\shuttleset_2.mp4",
            "video_url": "https://www.youtube.com/watch?v=-aOI9_JxoWc",
            "match_name": "CHEN Long CHOU Tien Chen World Tour Finals Group Stage",
            "fps": 25.0,
            "resolution": "1080p",
            "license_type": "MIT",
            "is_commercial": true,
            "status": "staging",
            "num_segments": 75
          },
          {
            "video_id": "shuttleset_3",
            "sport": "badminton",
            "csv": "badminton/shuttleset_3.csv",
            "local_path": "./data/videos\\shuttleset_3.mp4",
            "video_url": "https://www.youtube.com/watch?v=eugfCRwSBJo",
            "match_name": "Kento MOMOTA CHOU Tien Chen KOREA OPEN 2019 Final",
            "fps": 25.0,
            "resolution": "1080p",
            "license_type": "MIT",
            "is_commercial": true,
            "status": "staging",
```

```
      "num_segments": 78
    },
    {
      "video_id": "shuttleset_4",
      "sport": "badminton",
      "csv": "badminton/shuttleset_4.csv",
      "local_path": "./data/videos\\shuttleset_4.mp4",
      "video_url": "https://www.youtube.com/watch?v=y6QbtrTV-K0",
      "match_name": "CHEN Long CHOU Tien Chen Denmark Open 2019 QuarterFinal",
      "fps": 25.0,
      "resolution": "1080p",
      "license_type": "MIT",
      "is_commercial": true,
      "status": "staging",
      "num_segments": 74
    },
    {
      "video_id": "shuttleset_6",
      "sport": "badminton",
      "csv": "badminton/shuttleset_6.csv",
      "local_path": "./data/videos\\shuttleset_6.mp4",
      "video_url": "https://www.youtube.com/watch?v=SYyvDrUgClc",
      "match_name": "Kento MOMOTA CHOU Tien Chen Denmark Open 2018 Finals",
      "fps": 25.0,
      "resolution": "1080p",
      "license_type": "MIT",
      "is_commercial": true,
      "status": "staging",
      "num_segments": 112
    },
    {
      "video_id": "shuttleset_8",
      "sport": "badminton",
      "csv": "badminton/shuttleset_8.csv",
      "local_path": "./data/videos\\shuttleset_8.mp4",
      "video_url": "https://www.youtube.com/watch?v=32j2Tg64Zbg",
      "match_name": "CHOU Tien Chen Anders ANTONSEN Fuzhou Open 2019 Semi-finals",
      "fps": 25.0,
      "resolution": "1080p",
      "license_type": "MIT",
      "is_commercial": true,
      "status": "staging",
      "num_segments": 64
    },
    {
      "video_id": "shuttleset_9",
      "sport": "badminton",
      "csv": "badminton/shuttleset_9.csv",
      "local_path": "./data/videos\\shuttleset_9.mp4",
      "video_url": "https://www.youtube.com/watch?v=8E98Gpk-fOM",
      "match_name": "CHOU Tien Chen Jonatan CHRISTIE Sudirman Cup 2019 Quarter-finals",
      "fps": 25.0,
      "resolution": "1080p",
      "license_type": "MIT",
      "is_commercial": true,
      "status": "staging",
      "num_segments": 66
    },
    {
      "video_id": "shuttleset_10",
      "sport": "badminton",
      "csv": "badminton/shuttleset_10.csv",
      "local_path": "./data/videos\\shuttleset_10.mp4",
      "video_url": "https://www.youtube.com/watch?v=li1sbr6S34g",
      "match_name": "CHOU Tien Chen NG Ka Long Angus Sudirman Cup 2019 Group Stage",
```

```
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 72
        },
        {
          "video_id": "shuttleset_11",
          "sport": "badminton",
          "csv": "badminton/shuttleset_11.csv",
          "local_path": "./data/videos\\shuttleset_11.mp4",
          "video_url": "https://www.youtube.com/watch?v=yD6WKVqsAKc",
          "match_name": "CHOU Tien Chen Jonatan CHRISTIE Indonesia Open 2019 Quarter-
finals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 111
        },
        {
          "video_id": "shuttleset_12",
          "sport": "badminton",
          "csv": "badminton/shuttleset_12.csv",
          "local_path": null,
          "video_url": "https://youtube.com/watch?v=example",
          "match_name": "NG Ka Long Angus SHI Yu Qi Thailand Masters 2020 SemiFinals",
          "fps": 30.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 79
        },
        {
          "video_id": "shuttleset_13",
          "sport": "badminton",
          "csv": "badminton/shuttleset_13.csv",
          "local_path": "./data/videos\\shuttleset_13.mp4",
          "video_url": "https://www.youtube.com/watch?v=boQC4J4E1ZQ",
          "match_name": "Kento MOMOTA Viktor AXELSEN Malaysia Masters 2020 Finals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 78
        },
        {
          "video_id": "shuttleset_14",
          "sport": "badminton",
          "csv": "badminton/shuttleset_14.csv",
          "local_path": "./data/videos\\shuttleset_14.mp4",
          "video_url": "https://www.youtube.com/watch?v=5W6txLGZ1Rs",
          "match_name": "Anders ANTONSEN Jonatan CHRISTIE Indonesia Masters 2020
QuarterFinals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 97
        },
```

```
        {
          "video_id": "shuttleset_15",
          "sport": "badminton",
          "csv": "badminton/shuttleset_15.csv",
          "local_path": "./data/videos\\shuttleset_15.mp4",
          "video_url": "https://www.youtube.com/watch?v=yu9oyMXRGHY",
          "match_name": "Anthony Sinisuka GINTING Anders ANTONSEN Indonesia Masters 2020
Final",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 104
        },
        {
          "video_id": "shuttleset_16",
          "sport": "badminton",
          "csv": "badminton/shuttleset_16.csv",
          "local_path": "./data/videos\\shuttleset_16.mp4",
          "video_url": "https://www.youtube.com/watch?v=5kS_a7vS5xI",
          "match_name": "Anthony Sinisuka GINTING Viktor AXELSEN  Indonesia Masters 2020
SemiFinals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 60
        },
        {
          "video_id": "shuttleset_17",
          "sport": "badminton",
          "csv": "badminton/shuttleset_17.csv",
          "local_path": "./data/videos\\shuttleset_17.mp4",
          "video_url": "https://www.youtube.com/watch?v=G9r400zkkz8",
          "match_name": "NG Ka Long Angus Jonatan CHRISTIE Malaysia Masters 2020
QuarterFinals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 96
        },
        {
          "video_id": "shuttleset_18",
          "sport": "badminton",
          "csv": "badminton/shuttleset_18.csv",
          "local_path": "./data/videos\\shuttleset_18.mp4",
          "video_url": "https://www.youtube.com/watch?v=8lHAsyRhYYQ",
          "match_name": "Viktor AXELSEN  SHI Yu Qi All England Open 2020 QuarterFinals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 64
        },
        {
          "video_id": "shuttleset_19",
          "sport": "badminton",
          "csv": "badminton/shuttleset_19.csv",
          "local_path": "./data/videos\\shuttleset_19.mp4",
          "video_url": "https://www.youtube.com/watch?v=YmW83aQFADg",
```

```
          "match_name": "Viktor AXELSEN CHEN Long Malaysia Masters 2020 QuarterFinals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 107
        },
        {
          "video_id": "shuttleset_20",
          "sport": "badminton",
          "csv": "badminton/shuttleset_20.csv",
          "local_path": "./data/videos\\shuttleset_20.mp4",
          "video_url": "https://www.youtube.com/watch?v=4e3JJ4rvT3Q",
          "match_name": "Viktor AXELSEN NG Ka Long Angus Malaysia Masters 2020 SemiFinals",
          "fps": 25.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 74
        },
        {
          "video_id": "shuttleset_22",
          "sport": "badminton",
          "csv": "badminton/shuttleset_22.csv",
          "local_path": "./data/videos\\shuttleset_22.mp4",
          "video_url": "https://www.youtube.com/watch?v=8u_UHCnYSkk",
          "match_name": "Mia Blichfeldt Busanan Ongbamrungphan YONEX Thailand Open 2021
QuarterFinals",
          "fps": 30.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 70
        },
        {
          "video_id": "shuttleset_23",
          "sport": "badminton",
          "csv": "badminton/shuttleset_23.csv",
          "local_path": "./data/videos\\shuttleset_23.mp4",
          "video_url": "https://www.youtube.com/watch?v=rSK9Qx8LapE",
          "match_name": "Ng Ka Long Angus Lee Cheuk Yiu YONEX Thailand Open 2021
QuarterFinals",
          "fps": 30.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
          "num_segments": 71
        },
        {
          "video_id": "shuttleset_24",
          "sport": "badminton",
          "csv": "badminton/shuttleset_24.csv",
          "local_path": "./data/videos\\shuttleset_24.mp4",
          "video_url": "https://www.youtube.com/watch?v=NlDrJyQUTSI",
          "match_name": "Anthony Sinisuka Ginting Rasmus Gemke YONEX Thailand Open 2021
QuarterFinals",
          "fps": 30.0,
          "resolution": "1080p",
          "license_type": "MIT",
          "is_commercial": true,
          "status": "staging",
```

```
330           "num_segments": 99
331         },
332         {
333           "video_id": "shuttleset_25",
334           "sport": "badminton",
335           "csv": "badminton/shuttleset_25.csv",
336           "local_path": "./data/videos\\shuttleset_25.mp4",
337           "video_url": "https://www.youtube.com/watch?v=FZPrpoGdyHI",
338           "match_name": "Carolina Marin Supanida Katethong YONEX Thailand Open 2021
QuarterFinals",
339           "fps": 30.0,
340           "resolution": "1080p",
341           "license_type": "MIT",
342           "is_commercial": true,
343           "status": "staging",
344           "num_segments": 74
345         }
346       ]
347     }
```

13. assistant (2026-06-10T06:10:21.164Z)

The manifest matches the claim exactly. Now let me check how `is_commercial` gets derived, whether this is committed to git, and whether docs already acknowledge this as a known issue.

14. user (2026-06-10T06:10:21.654Z)

```
1       import csv
2       import os
3       from typing import Tuple, List, Dict, Any, Optional
4       from src.parsers.base_parser import BaseParser
5       from src.core.parsing import safe_int as _safe_int, parse_time_string
6       from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
7       from src.core.schemas.badminton import BadmintonRally
8       from src.core.validator import DatasetValidator
9
10
11      class CoachAIBadmintonParser(BaseParser):
12          def __init__(
13              self,
14              start_buffer: float = 1.0,
15              end_buffer: float = 2.0,
16              validator: Optional[DatasetValidator] = None,
17          ):
18              """
19              Args:
20                  start_buffer: Seconds to subtract from first stroke (serve) time.
21                  end_buffer: Seconds to add to last stroke time.
22                  validator: Used to decide commercial-license safety from the config
23                      whitelist. Defaults to a DatasetValidator with the default config so
24                      license classification stays consistent across the codebase.
25              """
26              self.start_buffer = start_buffer
27              self.end_buffer = end_buffer
28              self.validator = validator or DatasetValidator()
29
30          def parse(self, filepath: str, **kwargs) -> Tuple[VideoMetadata,
List[BadmintonRally]]:
31              """
32              Parses a CoachAI/ShuttleSet CSV format.
33
34              Expected CSV Columns (any subset or mapped names):
35                  - set: Set number (1, 2, 3)
36                  - rally: Rally number
37                  - ball_round: Shot number within the rally (1, 2, 3...)
```

```
                - time: Hitting time of the shot (in seconds)
                - type: Stroke type (e.g., smash, lob, net...)
                - score_a / score_b: Current scores
            """
            if not os.path.exists(filepath):
                raise FileNotFoundError(f"File not found: {filepath}")

            # Extract video details from kwargs or name
            match_name = kwargs.get("match_name") or
os.path.basename(filepath).replace(".csv", "")
            video_id = kwargs.get("video_id") or match_name.lower().replace(" ", "_")
            video_url = kwargs.get("video_url") or "https://youtube.com/watch?v=example"
            license_str = kwargs.get("license") or "MIT"

            # Check if license matches commercial whitelist
            try:
                license_type = LicenseType(license_str)
            except ValueError:
                license_type = LicenseType.UNKNOWN

            # Group rows by set & rally
            # Key: (set_num, rally_num) -> List of rows (dict)
            rallies_data: Dict[Tuple[int, int], List[Dict[str, Any]]] = {}

            with open(filepath, mode='r', encoding='utf-8') as f:
                reader = csv.DictReader(f)
                # Normalize column names to lowercase/strip
                reader.fieldnames = [name.lower().strip() for name in reader.fieldnames] if
reader.fieldnames else []

                for row in reader:
                    # Resolve key columns (tolerate empty cells and float-strings)
                    set_num = _safe_int(row.get('set'), 1)
                    rally_num = _safe_int(row.get('rally'), 0)
                    ball_round = _safe_int(row.get('ball_round', row.get('ball_seq')), 1)

                    # Resolve time (safely parse float or HH:MM:SS strings)
                    raw_time = row.get('time', row.get('timestamp', row.get('sec', '0.0')))
                    time_val = parse_time_string(raw_time)

                    key = (set_num, rally_num)
                    if key not in rallies_data:
                        rallies_data[key] = []

                    rallies_data[key].append({
                        'ball_round': ball_round,
                        'time': time_val,
                        'type': row.get('type', ''),
                        'score_a': row.get('score_a', row.get('round_score_a', '0')),
                        'score_b': row.get('score_b', row.get('round_score_b', '0'))
                    })

            # First pass: build raw rally records (unbuffered stroke bounds) in chrono
order.
            raw_rallies: List[Dict[str, Any]] = []
            for key in sorted(rallies_data.keys()):
                rows_sorted = sorted(rallies_data[key], key=lambda x: x['ball_round'])
                if not rows_sorted:
                    continue
                first_stroke = rows_sorted[0]
                last_stroke = rows_sorted[-1]
                raw_rallies.append({
                    'first_time': first_stroke['time'],
                    'last_time': last_stroke['time'],
                    'ending_reason': last_stroke['type'] if last_stroke['type'] else None,
```

```
                    'shots_count': len(rows_sorted),
                    'score_before': f"{first_stroke['score_a']}-{first_stroke['score_b']}",
                    'score_after': f"{last_stroke['score_a']}-{last_stroke['score_b']}",
                })

            # Second pass: apply start/end buffers, then resolve any boundary where the
            # padding of two adjacent rallies would collide by meeting at the midpoint of
            # the gap between their nearest strokes. This keeps the buffers from ever
            # producing an artificial overlap (which the validator would otherwise reject).
            starts = [max(0.0, rr['first_time'] - self.start_buffer) for rr in raw_rallies]
            ends = [rr['last_time'] + self.end_buffer for rr in raw_rallies]
            for i in range(len(raw_rallies) - 1):
                if ends[i] > starts[i + 1]:
                    midpoint = (raw_rallies[i]['last_time'] + raw_rallies[i +
1]['first_time']) / 2
                    ends[i] = min(ends[i], midpoint)
                    starts[i + 1] = max(starts[i + 1], midpoint)

            rallies: List[BadmintonRally] = []
            for i, rr in enumerate(raw_rallies):
                start_time = starts[i]
                end_time = ends[i]
                # Guarantee a strictly positive duration even after clamping.
                if end_time <= start_time:
                    end_time = start_time + 0.001

                rallies.append(BadmintonRally(
                    video_id=video_id,
                    rally_number=i + 1,
                    start_time=start_time,
                    end_time=end_time,
                    score_before=rr['score_before'],
                    score_after=rr['score_after'],
                    shots_count=rr['shots_count'],
                    ending_reason=rr['ending_reason']
                ))

            # Populate VideoMetadata
            video_metadata = VideoMetadata(
                video_id=video_id,
                sport=SportType.BADMINTON,
                video_url=video_url,
                local_path=kwargs.get("local_path", None),
                license_type=license_type,
                license_url=kwargs.get("license_url", None),
                is_commercial=self.validator.check_license_commercial(license_type),
                status=CurationStatus.STAGING,
                match_name=match_name,
                fps=kwargs.get("fps", 30.0),
                resolution=kwargs.get("resolution", "1080p")
            )
            return video_metadata, rallies

```

15. user (2026-06-10T06:10:23.136Z)

```
(Bash completed with no output)
```

16. user (2026-06-10T06:10:33.451Z)

```
.gitignore:44:data/eval_export*/        data/eval_export/manifest.json
```

---- Python ----

```
__pycache__/
*.py[cod]
```

```
*$py.class
*.egg-info/
.eggs/
build/
dist/
.installed.cfg
*.egg
```

---- Test / coverage ----

```
.pytest_cache/
.coverage
.coverage.*
htmlcov/
.tox/
.cache/
```

---- Virtual environments ----

```
.venv/
venv/
env/
ENV/
```

---- Secrets / local config ----

```
.env
.env.*
```

---- OS / editor cruft ----

```
.DS_Store
Thumbs.db
.idea/
.vscode/
*.swp
```

---- Project: large media kept outside git (see README) ----
The video corpus is hundreds of MB+; only metadata (the SQLite DB) and the small source annotation CSVs are version-controlled. Re-fetch videos from the stored video_url / local_path per the manifest.

```
data/videos/
```

---- Generated eval/training exports (regenerated via `curate export`) ----

```
data/eval_export*/
```

---- Generated temp artifacts (crawler scratch space; regenerated each run) ----

```
data/temp/
```

---- Per-video observability reports (written next to videos / inputs at runtime) ----

```
*.report.json
*.report.md
*.summary.md
```

17. user (2026-06-10T06:10:33.651Z)

```
No files found
```

18. user (2026-06-10T06:10:41.058Z)

```
/c/Users/avidu/Projects/sports-data-collector/docs:
```

```
DECISIONS.md
INGESTION_PIPELINE.md
MANUAL_INGESTION_POSITIONING.md
PIPELINE_ROLE.md
annotation

/c/Users/avidu/Projects/sports-data-collector/docs/annotation:
ANNOTATION_GUIDE.md
INTAKE_MANIFEST_TEMPLATE.csv
NDA_TEMPLATE.md
ROORA_BRIEF.md
```

19. user (2026-06-10T06:10:48.987Z)

```
4:sports videos with per-rally `[start, end)` annotations and commercial-license
20:commercially-licensed, annotated corpus** ? making rally detection free, offline,
27:- **`curate export`** ? emits the indexer's ground-truth format: per-video
31:  `is_commercial`, so only licensable data flows
38:  hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
```

20. user (2026-06-10T06:10:50.087Z)

```
No matches found
```

21. user (2026-06-10T06:10:55.230Z)

```
1       # Role in the rally-segmenter training pipeline
2
3       This repo is the **data producer** in a two-repo system. Its job is to curate
4       sports videos with per-rally `[start, end)` annotations and commercial-license
5       metadata, then hand that data to the **badminton-highlight-indexer**, which
6       processes, scores, and (Phase 4) *trains* an offline rally segmenter on it.
7
8       This doc explains *why* the `export` and `fetch` commands exist and how they fit
9       the bigger picture. The full decision records and end-to-end narrative live in the
10      indexer repo:
11
12      - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13      - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14        (acquisition policy), ADR-004 (offline learned segmenter).
15
16      ## Why this repo matters
17
18      The indexer's best rally mode refines local detections with a **paid Gemini API**.
19      The strategic goal is to replace that with an **offline model trained on our own
20      commercially-licensed, annotated corpus** ? making rally detection free, offline,
21      and self-improving. None of that is possible without a clean, licensable,
22      accurately-labelled, full-resolution dataset. That dataset is what this repo
23      produces.
24
25      ## What the two key commands do (and why)
26
27      - **`curate export`** ? emits the indexer's ground-truth format: per-video
28        `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29        Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30        table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31        `is_commercial`, so only licensable data flows
32        downstream. The annotations are **temporal-only** ? they train/evaluate rally
33        *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35      - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36        the DB **in place** so annotations and curation status are never lost. Uses
37        authenticated cookies for reliable, unthrottled downloads. The why and the
38        hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
```

```
39
40      ## The one contract that matters: video identity
41
42      The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43      re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44      exact `local_path`, and any downstream processing must run on **that** file. This is
45      why acquisition (`fetch`) happens *before* the indexer processes anything.
46
```

22. user (2026-06-10T06:10:56.476Z)

```
/c/Users/avidu/Projects/sports-data-collector/docs/DECISIONS.md
72 /c/Users/avidu/Projects/sports-data-collector/docs/DECISIONS.md
```

23. user (2026-06-10T06:11:06.557Z)

```
Found 6 files
docs\GOLDEN_SET_VENDORING.md
docs\archives\decisions\DECISIONS.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\ROADMAP.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\GOLDEN_SET_IMPLEMENTATION.md
```

24. user (2026-06-10T06:11:07.394Z)

```
Found 11 files
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\GOLDEN_SET_VENDORING.md
docs\BACKLOG.md
docs\COMMERCIALIZATION.md
docs\archives\decisions\DECISIONS.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\PMF_MARKET_RESEARCH.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
```

25. user (2026-06-10T06:11:13.155Z)

```
110-### Context
111-
112-The collector's initial crawl downloaded videos with `yt-dlp -f worstvideo` (256×144),
113-mislabelled as 1080p. The indexer needs ?720p MP4 for meaningful YOLO/shuttle detection.
114-Re-acquiring full-resolution sources surfaced several hard-won realities:
115-
116:- The ShuttleSet corpus **tops out at 1080p** (no 4K); "best available" resolves to 1080p.
117:- Authenticating with a **YouTube Premium** account is required for reliable, unthrottled
118-  downloads ? anonymous downloads are throttled (~1.3 MiB/s) and **non-deterministic**
119-  (identical commands variously yielded 1080p DASH, 1080p HLS, or a 144p fallback).
120:- **Chrome** cookies can't be read on Windows (app-bound encryption / DPAPI, yt-dlp
121-  #10927); **Firefox** cookies read cleanly even while open.
122:- The `web`/`web_safari` player clients are forced into **SABR streaming**, which strips
--
137-4. **Throttle bulk runs** (`--sleep` + per-video `--retries`/backoff) and treat a
138-   below-min-height download as a **failure** (discard, don't commit) so a flaky low-res
139-   result never overwrites a good entry.
140-
141-### Consequences
142-
143:- All 22 fetchable ShuttleSet matches are now true 1080p (one match, shuttleset_12, has no
144-  source URL in the catalog and remains video-less but keeps its annotations).
145:- Acquisition is reproducible and resumable; the ~26 GB corpus is kept outside git.
```

```
146-- Future non-YouTube or 4K sources are supported by the same code (optional `max_height`).
147-
148----
149-
--
195-  tennis/cricket once those have annotations.
196-- The full paid 22-video Gemini evaluation is **not** run; spend is limited to a single
197-  reference video.
198-
199-### Update (2026-05-26): Phase 3 evidence ? substrate is WASB, not YOLO
200-
201:The deciding candidate-recall measurement is in. On shuttleset_8 (64 GT rallies), pure
202-YOLO Stage-1 produced 181 candidate windows with:
203-
204-- **Recall @ IoU 0.5 = 0.05**, but recall @ IoU 0.1 = 0.53 and @ IoU 0.01 = 0.72.
205-- **72%** of rallies have *some* candidate overlap, yet the **median best coverage of a
206-  rally by any candidate is only ~20%**.
207-- Post-hoc **merging adjacent candidates does not help** (recall stays ~0.05 and degrades
--
267-Decisions locked with the owner: **licensed/owned video only** (no user uploads to vendors)
· **rally `[start,end)` annotation depth** · **tiered turnaround** (fast automated + slow human)
· **vendor bake-off** to choose · **Bangalore/India-local preference**.
268-
269-### Decision
270-
271-1. **One scorer, three jobs.** A vendor's `[start,end)` output is structurally identical to
a prediction, so `metrics.py` grades product-vs-truth, vendor-vs-key, and annotator-vs-annotator
(IAA) with no new scoring code.
272-2. **Select vendors by measured quality-per-dollar**, via a bake-off: write a precise rally-
definition guideline, build a small internal gold key (5?10 licensed clips), send the same clips
to 2?3 vendors (favoring Bangalore/India), and grade every submission with our own harness. The
numbers are the decision.
273:3. **Pluggable `AnnotationProvider` registry** mirroring the existing registry pattern:
`FileProvider` (today's manual/ShuttleSet flow), `HumanVendorProvider` (slow tier ?
authoritative truth), `AutomatedProvider` (fast tier ? independent model oracle).
274-4. **Regression = existing harness + one new component.** `regress(video)` calls a provider
for ground truth, runs the production pipeline, and scores with the unchanged `metrics.py`
against a per-video `baseline.json`.
275-5. **Privacy by construction.** Golden sets use only licensed/owned footage; all third-party
video egress flows through the single provider boundary where DPA / no-reuse / delete-on-
delivery / access-logging controls live.
276-
277-### Consequences
278-
279-- Eliminates the "human views user video" concern for golden-set/regression work, because
user uploads never enter the pipeline.
```

26. user (2026-06-10T06:11:22.525Z)

```
No matches found
```

27. user (2026-06-10T06:11:22.951Z)

```
No files found
```

28. user (2026-06-10T06:11:24.918Z)

```
docs\COMMERCIALIZATION.md:191:2. **Per-checkpoint / dataset licenses** for WASB / TrackNetV3 /
any VLM ? train-own or
docs\BACKLOG.md:17:- ? **Per-checkpoint weight/dataset licenses** ? if we borrow pretrained
weights (YOLOX, ActionFormer, WASB, TrackNetV3), the *code* license ? the distributed
*weights/dataset* terms. Either train our own or confirm each checkpoint. **Revisit:** when
adopting any borrowed weights.
docs\GOLDEN_SET_VENDORING.md:30:- **Golden sets are built only from footage we license or own**
```

? starting with the founder's **constantly-growing personal collection** of match videos, plus ShuttleSet (ADR-003) and any purpose-recorded clips. **No end-user uploads ever flow to a vendor.**
docs\archives\MONETIZATION_AUDIT.md:111:2. **Pretrained-weights / dataset licenses** for YOLOX, ActionFormer, TrackNetV3, WASB ? repos state a *code* license but don't separately license distributed checkpoints. Train-own, or confirm each checkpoint.
docs\ROADMAP.md:24:per-rally `[start, end)` annotations and a commercial-license flag. The thesis of
docs\archives\decisions\DECISIONS.md:267:Decisions locked with the owner: **licensed/owned video only** (no user uploads to vendors) · **rally `[start,end)` annotation depth** · **tiered turnaround** (fast automated + slow human) · **vendor bake-off** to choose · **Bangalore/India-local preference**.
docs\archives\decisions\DECISIONS.md:275:5. **Privacy by construction.** Golden sets use only licensed/owned footage; all third-party video egress flows through the single provider boundary where DPA / no-reuse / delete-on-delivery / access-logging controls live.

29. user (2026-06-10T06:11:35.598Z)

No matches found

30. user (2026-06-10T06:11:37.409Z)

49- AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later),
50- and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So: **head now ? VLM later**,
51- strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes, head refines boundaries) over
52: wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur footage may differ ? re-measure.)*
53-
54-## Conversational video-QA ? build the bookkeeping NOW, punt the model
55-**Punt the conversational *feature*** until extraction quality is solid (current LOVO ~0.583 ? "very high"; a chat UI

31. user (2026-06-10T06:11:46.065Z)

4:milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a training source).
5:**Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO + learned prototype
15:- **Immediate deliverable:** a **training framework + harness** that produces a model with **very high precision/
27:- **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public grounding-CoT datasets that
29:- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by match.**
30:- ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl. Qwen3-VL).** Our "no paid-LLM
31: training" rule is enforceable at *our fine-tuning data* layer, not the base's pretraining ? an **explicit owner
47: (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries").
48: **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-risked on our corpus (held-out
49: AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later),
63: corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags,
69: it reasons over our data, it is not trained on its outputs).
73: feature / per-sport calibration). The collector + indexer are already sport-generic. ? **ACTION: rename the repo off
80:- ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not proven ? our corpus is badminton-only,

```
83:  corpus exists. Don't cite RacketVision as evidence.
86:- **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
90:- **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate our
dev language:** train in
92:  runs the artifact with zero Python. **The one concrete constraint now: train with export-
clean ops** so the compression
101:### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) · *greenlight
item*
102:Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index schema
(the conversational seam)
104:shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
106:one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector` (system-
of-record; extends PR #13).
107:**Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match enforced;
**no Gemini-sourced rows in any training view**.
109:### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
114:### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) · *greenlight
item*
115:- **(i) PURGE the Gemini-derived TRAINING labels** in `training.label_candidates` (over
Gemini CSVs) ? a **live
116:  no-paid-LLM-training violation** that bakes in the moment a training run precedes it.
Retarget to human + open-teacher
119:  blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean detector swap; **multiplies
123:Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
127:F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory. Trains
on the GPU/WSL box.
135:## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus + explicit
go)
140:  our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on Modal
(no-access) / RunPod-Secure.
150:| 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
153:| 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159:trajectory detectors not yet identified · base-model pretraining provenance unauditable
(risk-acceptance) · real VLM
160:VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-train
(minors-excluded) for cloud
161:training · on-device 4B latency on 8 GB unmeasured.
```

32. user (2026-06-10T06:11:47.337Z)

```
3-This document is the **narrative thread**: how the indexer evolves from an
4-AI-API-dependent prototype into a self-contained tool that learns rally detection
5:from its own commercially-licensed, annotated video corpus. It explains *how we
6-got here*, *why each step matters*, and *how it shapes the final tool*. Decisions
7-are recorded as ADRs in [DECISIONS.md](DECISIONS.md); the scoring mechanics live
--
17-
18-> *Naming note:* the `yolo_hybrid` registry key is retained for back-compat, but
19:> AGPL YOLOv8 was removed and replaced with a BSD/MIT-licensed torchvision +
20-> ByteTrack stack ? see **ADR-005**. "YOLO" in this doc means the local
21-> player-motion gating stage, not the Ultralytics library.
22-
23-A sibling project, **sports-data-collector**, curates sports videos with
24:per-rally `[start, end)` annotations and a commercial-license flag. The thesis of
25:this roadmap: **use that annotated, commercially-usable data to (1) measure the
26-pipeline objectively and (2) train an offline replacement for the paid Gemini
27-stage** ? turning the tool into one that improves as more data is curated, runs
--
37-A sport-agnostic `curate export` in the collector emits the indexer's `start,end`
38-ground-truth CSVs plus a `manifest.json` pairing each CSV with its video, gated on
```

```
39:the `is_commercial` flag. ? *Why:* one clean producer/consumer contract that works
40-for all five supported sports (badminton/tennis/cricket/pickleball/table-tennis) alike.
41-See **ADR-002**.
--
95-- **Sport-generic.** The label ? feature ? classifier machinery and the export
96-  bridge already span sports; adding a sport is data, not new architecture.
97:- **Commercially clean.** Only `is_commercial` data flows through the bridge, so the
98-  trained model and any derived product rest on a licensable foundation.
99-
```

33. user (2026-06-10T06:11:57.899Z)

```
1       # Decision records
2
3       Lightweight ADRs for this repo. (The cross-repo training-pipeline ADRs live in
4       the indexer repo's `docs/DECISIONS.md`; this file is for collector-local choices.)
5
6       ---
7
8       ## ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)
9
10      **Status:** Accepted · **Date:** 2026-06-02
11
12      ### Context
13      Supporting a sport currently requires parallel edits across several sites:
14      - `SEGMENT_TABLES` + a `CREATE TABLE` + `insert_*` / `get_*` in `src/core/db.py`
15      - a `validate_*` method in `src/core/validator.py`
16      - `import-local` parse/validate/insert branches and interactive review/play in
`src/cli/curate.py`
17      - a schema in `src/core/schemas/`
18
19      A code review flagged this as copy-paste duplication and proposed a registry/plugin
20      mechanism (e.g. `SPORT_REGISTRY: dict[SportType, SportSpec]` driving generic
21      insert/get/validate/dispatch) to make adding a sport a single entry.
22
23      ### Decision
24      **We are keeping the explicit per-sport code and NOT introducing a registry
25      abstraction at this time.**
26
27      ### Rationale
28      - **It's well-tested and mechanical.** Coverage is ~87%; adding a sport is a known,
29        ~30-minute "mirror an existing sport" task (checklist below), not a research problem.
30      - **Schemas are heterogeneous.** Rally sports (badminton, pickleball, table tennis)
31        share `rally_number / shots_count / ending_reason`, but tennis (`set_score`,
32        `game_score`, `server`, `ending_type`) and cricket (`over`, `ball`, `runs`,
33        `wicket`, `extras`) differ enough that a single generic table/row mapper would leak
34        special-cases ? trading visible duplication for hidden complexity.
35      - **Explicit code is greppable.** Contributors can follow one sport end-to-end without
36        learning a registry indirection. Premature abstraction is harder to undo than dup.
37      - **Low, slow growth.** Five sports today; new ones arrive rarely.
38
39      ### Consequences
40      - Adding a sport touches ~5 sites (see checklist). We accept that boilerplate.
41      - If the cost rises, ADR-001 should be revisited and superseded.
42
43      ### Revisit triggers (when to build the registry)
44      - Sport count grows beyond ~8, **or**
45      - We add a 4th+ rally-identical sport and the dispatch churn starts causing bugs, **or**
46      - A new cross-cutting feature would otherwise require editing every per-sport branch.
47
48      ### Future design sketch (when revisited)
49      ```
50      @dataclass
51      class SportSpec:
52          schema_class: type
```

```
          table_name: str
          pk_columns: tuple[str, ...]
          columns: tuple[str, ...]
          validate_fn: Callable

      SPORT_REGISTRY: dict[SportType, SportSpec] = { ... }
      # Generic: insert_segments(sport, items) / get_segments(sport, video_id) /
      #          validate(sport, items) / import-local dispatch driven by the registry.
      ```

      ## Checklist: adding a new sport today (under ADR-001)

      1. `src/core/schemas/<sport>.py` ? define the segment model (mirror the closest existing
sport) with a `start_time < end_time` validator.
      2. `src/core/db.py` ? add to `SEGMENT_TABLES`, add a `CREATE TABLE`, `insert_<sport>()`,
`get_<sport>()`, and a cascade `DELETE` in `delete_video`.
      3. `src/core/validator.py` ? add `validate_<sport>()` (duration + no-overlap).
      4. `src/cli/curate.py` ? add branches in `import_local` (validate + insert),
`_parse_local_annotations` (JSON **and** CSV), and the interactive review/play.
      5. `config/config.yaml` ? add the sport to `supported_sports` (and confirm the
`SportType` enum value in `schemas/base.py`).
      6. `tests/` ? mirror the existing db/validator/cli tests for the new sport.
```

**34. user (2026-06-10T06:11:59.353Z)**

```
1-# Sports Data Collector & Curator
2-
3:A clean, extensible Python codebase and CLI toolchain designed to discover, ingest, validate,
and curate sports video datasets. The repository stores annotations and video metadata in an
SQLite database, supports sport-specific tables and schemas, manages video storage outside of
Git, and enforces strict commercial license compliance.
4-
5:> **Pipeline role:** this repo is the *data producer* feeding the **badminton-highlight-
indexer**, which trains/evaluates an offline rally segmenter on this curated, commercially-
licensed data. See docs/PIPELINE_ROLE.md for why `export`/`fetch` exist
and how the two repos fit together.
6-
7----
--
10-
11-- **SQLite Backend**: Centralized `sports_metadata.db` with a generic `videos` table and
sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`,
`pickleball_rallies`, `tabletennis_rallies`).
12:- **Commercial License Flagging**: Each ingested source license is checked against a
configured whitelist (`MIT`, `Apache-2.0`, `BSD`, `CC0`, `CC-BY`, `Public-Domain`,
`Proprietary`) and the resulting `is_commercial` flag is stored alongside the video. Non-
commercial sources (like `CC-BY-NC-SA`) are flagged as non-commercial but still ingested ?
nothing is discarded automatically. The whitelist is read from `config/config.yaml` consistently
across the parser, validator, and CLI.
13-- **Overlap & Duration Validation**: Ensures chronological event bounds (e.g. rally start/end
times) do not overlap and that durations are positive.
14-- **Stroke-level CSV Parser**: Ingests ShuttleSet/CoachAI-style stroke CSV logs and
aggregates them into rally durations using configurable start/end time buffers.
--
63-To parse external raw stroke-level datasets (e.g. CoachAI format) and register them in
staging:
64-```bash
65:python -m src.cli.ingest --sport badminton --parser coachai --file-path
data/staging_coachai_match.csv --license CC-BY-NC-SA --video-id staging_match_non_comm
66-```
67-
--
```
```

```
75-To register a local video file alongside its JSON\CSV annotation file (bypasses staging):
76-```bash
77:python -m src.cli.curate import-local --sport badminton --video-path
"data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json"
--match-name "My Custom Local Match" --license Proprietary
78-```
79-`import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`,
`pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the
canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g.
via an annotation vendor), see [`docs/annotation/`](docs/annotation/).
--
88-python -m src.cli.curate fetch --video-id shuttleset_1 --force --cookies-from-browser firefox
89-```
90:[Omitted long matching line]
91-
92-> **Authenticated cookies are strongly recommended.** Anonymous YouTube downloads are
throttled and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser
firefox` (or `chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a
logged-in (ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-
bound cookie encryption may block direct reads on Windows; Firefox reads cleanly even while
open.
--
97-To turn curated annotations into ground-truth files the badminton-highlight-indexer can score
and train against, export per-video `start,end` CSVs plus a `manifest.json`:
98-```bash
99:# Curated + commercially-safe videos across all supported sports (default):
100-python -m src.cli.curate export --out-dir data/eval_export
101-
--
103-python -m src.cli.curate export --sport badminton --include-staging
104-```
105:[Omitted long matching line]
106-
107----
--
111-Adjust settings in `config\config.yaml`:
112-- **`database_path`**: Path to the SQLite database.
113:- **`video_storage`**: Local folder path and automatic download size limit.
114:- **`commercial_licenses`**: Whitelist of licenses allowed for commercial use.
115-- **`supported_sports`**: Allowed sports categories.
116-
```

35. user (2026-06-10T06:12:15.813Z)

```
93:   file recorded in the manifest; acquisition (ADR-003) therefore precedes processing.
105:## ADR-003: Video acquisition ? always best resolution, upgrade in place, authenticated
cookies
```

36. user (2026-06-10T06:12:17.111Z)

```
No matches found
```

37. user (2026-06-10T06:12:28.610Z)

```
124:- **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
```

38. user (2026-06-10T06:12:29.951Z)

```
138:[Omitted long matching line]
```

39. user (2026-06-10T06:12:43.988Z)

```
136     - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
```

stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid` clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
137   - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels** (PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ? `window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ? P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate + metrics.
138   - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter = owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean, spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO. `::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/ calibrate_local.
139   - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court isolation** (spatial gate to the prominent court, needs real court-region detection). Academy = multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140   - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT: duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal), `::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`), `::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-one-video-out needs ?2 videos; final model trained on all.

40. user (2026-06-10T06:13:39.256Z)

Structured output provided successfully

41. assistant (2026-06-10T06:13:45.324Z)

Verification complete. The finding is real with high confidence ? all cited code, schema, gate, and manifest claims verified verbatim, with two factual corrections (manifest is gitignored rather than committed; 22/24 entries are staging-status and no training run has consumed the corpus yet).